

tp-sapu

Temporal Parallel Spiking Adaptive Processing Units

Architecture · MNIST sanity check · Tiny ImageNet · Attention analysis · Future hardware

Mateusz — June 2026

Why Not Just Use an RNN or a CNN?

Standard CNN

- Dense float multiply-adds
- **No time**: processes whole image at once
- Strong spatial inductive bias
- Energy $\propto$ FLOPs $\times$ precision

Vanilla RNN/LSTM

- Sequential token loop — **similar**
- Dense
$$h_t = \tanh(Wx_t + Uh_{t-1})$$
- One hidden state, one timescale
- Vanishing gradient / slow memory

TP-SAPU SNN

- **Binary spike outputs** — multiply $\rightarrow$ gate
- **Multiple τ** = free multi-scale memory
- $\geq 95\%$ **sparse** recurrent weights
- Shared W_{rec} across all timescales

Key difference from RNNs

An LSTM uses one dense hidden state updated with tanh/sigmoid gates. TP-SAPU runs N_τ **parallel LIF reservoirs**, each with a different leak constant τ , all sharing **the same W_{rec}** . Result: N_τ timescales at the cost of one recurrent matrix, and spikes instead of continuous activations.

The LIF Neuron: Integration, Threshold, **Sparse Spike**

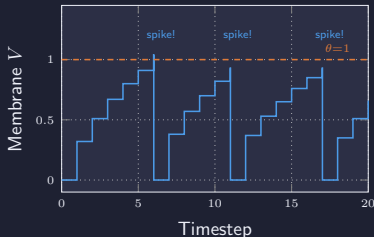
Membrane dynamics per timestep t :

$$V_t = V_{t-1} + \underbrace{(W_{\text{in}}x_t + W_{\text{rec}}s_{t-1})}_{\text{current}} - V_{t-1} \cdot \frac{1}{\tau}$$

$$s_t = \begin{cases} 1 & V_t \geq \theta \\ 0 & \text{otherwise} \end{cases} \quad V_t \leftarrow 0 \text{ if } s_t=1$$

- τ small $\Rightarrow$ fast leak, reacts token-by-token
- τ large $\Rightarrow$ slow leak, integrates context
- **Output** $s_t \in \{0, 1\}$ — a *spike*, not a float
- Surrogate gradient (ATan) enables backprop through threshold

Typical membrane trace ($\tau=4$)



Spike rate $\approx$ 5–15% per step in practice.
95%-sparse pruning: only 5% of W_{rec} non-zero.
Combined: $\ll$ 1% of multiplications needed.

Why sparsity matters

In hardware, $s_t=0$ means **no multiply**. With 95% sparse W_{rec} and $\sim$ 10% spike rate, $\sim$ 99% of multiplications become no-ops — free hardware efficiency.

TP-SAPU Backbone: One Matrix, Many Timescales

Core idea: shared topology, different τ

- 1 Input token x_t projected: $z_t = W_{in}x_t$
- 2 For each of N_τ reservoirs in parallel:
 $V_t^\tau += (z_t + W_{rec}s_{t-1}^\tau - V_t^\tau)/\tau$
- 3 Spike: $s_t^\tau = 1[V_t^\tau \geq 1]$, reset $V_t^\tau \leftarrow 0$
- 4 Output: $\text{cat}(V_t^{\tau_1}, \dots, V_t^{\tau_N})$ **per step**

Parameter sharing

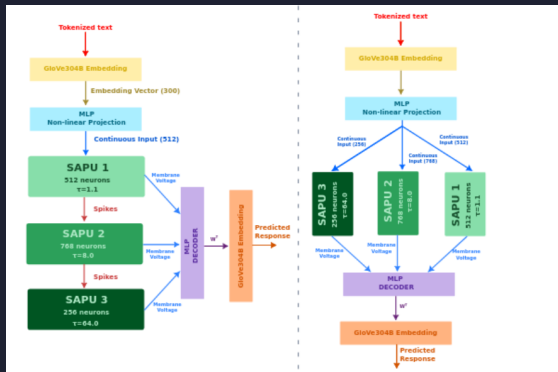
$W_{in} \in \mathbb{R}^{d \times d}$, $W_{rec} \in \mathbb{R}^{d \times d}$ are **shared** across all N_τ reservoirs.

MNIST: $d=64$, $N_\tau=3 \Rightarrow 192$ features/step.

ImageNet: $d=64$, $N_\tau=8 \Rightarrow$ **512 features/step**.

Sparsity schedule

L2-driven magnitude pruning ramps from 0 to **95%** over 24 epochs. Final 6 epochs stabilize at the fixed mask. Only 5% of recurrent weights remain non-zero.



Overview of LLM versions of SAPU, both parallel and serial.

End-to-End Pipeline: Encoder $\rightarrow$ Backbone $\rightarrow$ Decoder

The three stages:

Encoder — image $\rightarrow$ token sequence

cnn3: $\text{Conv}(s=2) \rightarrow \text{Conv}(s=2) \rightarrow \text{Conv}(s=1)$

On 64×64 : $(B, 3, 64, 64) \rightarrow (B, 256, 128)$ — **256 tokens, 16×16 grid**

res_cnn: same + residual, concatenate both layers $\rightarrow$ 512 tokens

Backbone — tokens $\rightarrow$ reservoir states

Each token is one LIF step for all N_τ reservoirs in parallel.

Output (membrane, 'pooling=none'): $(B, 256, N_\tau \cdot d_{\text{res}})$, e.g. $(B, 256, 512)$

Decoder — states $\rightarrow$ logits

MLP variants: pool last/mean, classify

Transformer: LayerNorm+Linear $\rightarrow$ CLS+256 positions $\rightarrow$ 2-layer encoder $\rightarrow$ classify

model_dim=128, 4 heads, 512-FFN, **bidirectional**, pooling=none



	MNIST	TinyIM (sm)	TinyIM (lg)
Image	28^2	64^2	64^2
Tokens	49	256	512
N_τ	3	8	8
d_{res}	64	64	256
Neurons	192	512	2048
Params	$\sim 200\text{k}$	702k	1.65M

MNIST Sanity Check: 42 Encoder×Decoder Combinations

Encoder	linear	mem_mlp	spike_mlp	both_mlp	all_state	lif_count
cnn3	99.4	99.3	98.7	99.4	99.4	99.3
res_cnn	99.3	99.3	98.7	99.4	99.4	99.1
cnn2	99.1	99.1	96.9	99.2	99.2	99.0
mlp_pch	98.4	98.6	84.9	98.6	98.7	98.0
rows	97.0	98.3	94.0	98.4	98.4	96.8
lin_pch	97.4	98.1	90.1	97.9	98.3	97.0
lif_2x2	91.3	93.4	61.7	92.6	95.1	90.2

Best: **99.41%** (cnn3 + both_mlp or all_state_mlp).
spike_mlp always worst — raw binary outputs too noisy.
20 epochs, no pruning, ~200k params.

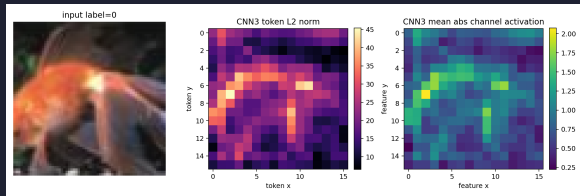
Observations:

- **Encoder quality dominates:** CNN > patch > rows
- lif_2x2 severely limited — biologically motivated but 196 weak binary tokens hurt
- Combining membrane + spike (both_mlp) **always beats single modality**
- Even a linear decoder over TPSAPU membranes achieves >97%
- Architecture is robust — 35 of 42 combinations exceed 96%

Sanity check passed

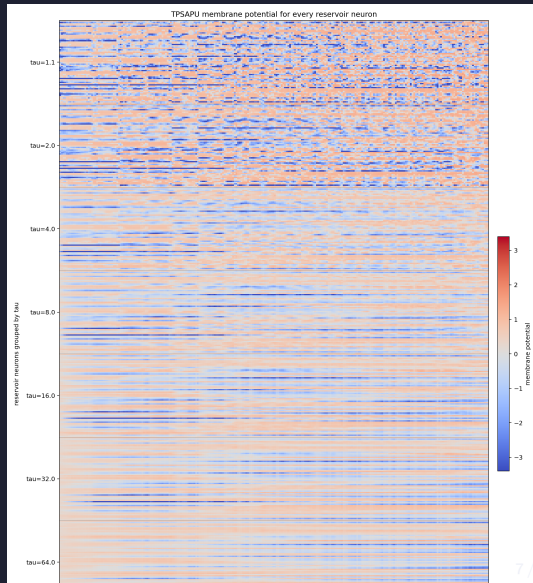
TPSAPU is able to learn on MNIST correctly, despite no augmentation. Most models converged around epoch 13.

What the Reservoir Sees: Tokens & Voltage Matrix

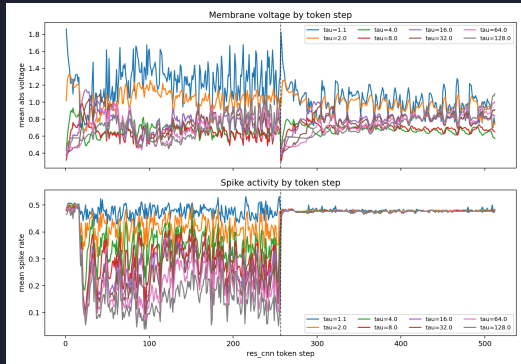


Left: input Centre: token L2-norm heatmap Right: channel activations

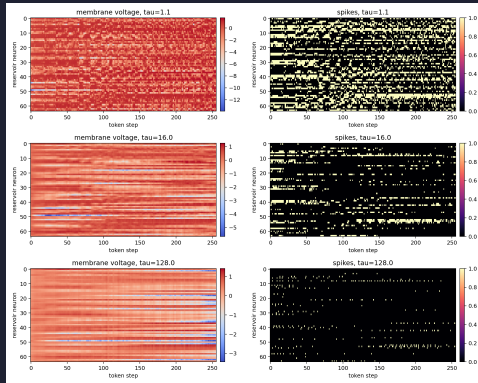
- CNN concentrates high-norm tokens on **object edges**
- Background patches $\approx$ zero norm — sparse input
- The backbone sees this as a **sequence of timesteps**



Temporal Dynamics: Per- τ Traces & Spike Heatmaps



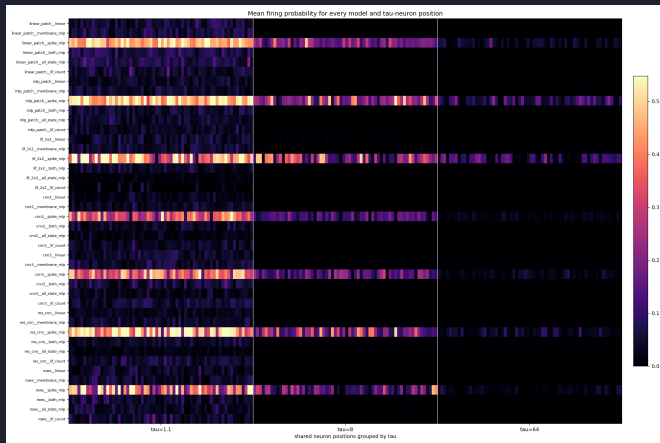
Mean membrane voltage (solid) and spike rate (dashed) per τ , over token steps.



Per-neuron membrane (left) and spikes (right) for selected τ values.

Multi-timescale hypothesis: fast reservoirs ($\tau \approx 1.1$) react sharply to edges and local features. Slow reservoirs ($\tau=128$) build a global summary over the whole sequence. The decoder sees *all* τ -bands simultaneously — a richer feature than any single timescale. This is what differentiates TP-SAPU from a single-state RNN: **multiple temporal abstractions at no extra recurrent cost.**

MNIST: Spiking Rates & Spectral Diagnostics



How to read the spectrum plots:

Stable rank says how many independent directions the weights really use. Higher means the model is not relying on one dominant shortcut.

Spectral radius says how strongly recurrent activity can be amplified. Too small forgets quickly; too large can become unstable or overly reactive.

Non-normality means weights can temporarily amplify signals even if they look stable from eigenvalues alone. In ML terms: the dynamics may create short bursts of useful activity, but can also become harder to decode.

Spike-only models learn to make neurons fire, but membrane readout still wins because voltage keeps weak evidence before a spike happens.

MNIST Analysis: Insights & Gaps

What drives performance

- 1 **Encoder quality** — CNNs beat patch, patch beats rows
- 2 **Combined readout** — both/all-state beats single modality
- 3 **Timescale diversity** — 3 taus better than 1

What doesn't help

- Spike-only decoders — too noisy for classification, but shows that we can train neurons to spike no matter the tau.
- lif_2x2 encoder — strong biological motivation, poor results
- Larger reservoirs on MNIST ($d_{\text{res}} = 64$ is sufficient)

Missing experiments

- Transformer decoders on MNIST — experimented with one encoder and converged to 99.5% almost instantly, but I didn't do a full study.
- No pruning applied (`prune_epochs = 0`) — can we achieve 95% sparsity on MNIST with every combination?
- No documented tau ablation — how much do you lose with $N_\tau = 1$?
- Backbone baseline not documented, but most smaller pairs stayed at $\sim 15\%$, cnn based converged to around 95%.

Verdict: TPSAPU achieves MNIST accuracy (99.4%) at $\sim 200k$ params with sparse binary processing. The core architecture works. Scaling behaviour on harder tasks is the next question.

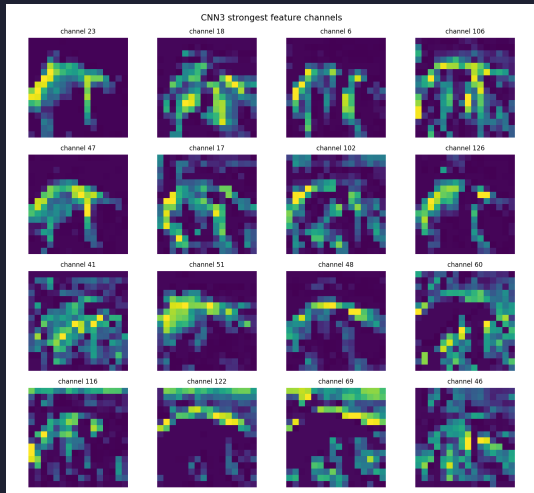
CNN3 Encoder: How the Image Becomes a Token Sequence

Three convolutions, then flatten:

Layer	Output shape	Covers
Input	$(B, 3, 64, 64)$	full image
Conv1 ($s=2$)	$(B, 64, 32, 32)$	2×2 px
Conv2 ($s=2$)	$(B, 64, 16, 16)$	4×4 px
Conv3 ($s=1$)	$(B, 128, 16, 16)$	4×4 px
flatten+pos	$(B, \mathbf{256}, 128)$	16×16 grid

- 256 tokens — each covers a **4×4 pixel block**
- Learnable positional embedding ($\sigma = 0.02$)
- `patch_size=8` in args is **ignored** by CNN encoders — grid size comes from convolution strides only
- GELU activations, light Dropout2d(0.05)

The TPSAPU backbone then processes the 256 tokens **one step at a time**, treating the spatial raster-scan order as a temporal sequence.



Top CNN feature channels by mean absolute activation (validation sample 0).

Transformer Decoder: How the CLS Token Reads the Reservoir

Pipeline for membrane_transformer (702k):

Step	Shape
TPSAPU output (membrane)	$(B, 256, 512)$
LayerNorm + Linear(512→128)	$(B, 256, 128)$
Prepend CLS token	$(B, 257, 128)$
+ learned pos. embed	$(B, 257, 128)$
×2 TransformerEncoderLayer	$(B, 257, 128)$
4 heads × 32 dims, 512-FFN	
Post-LN, GELU, bidirectional	
Extract CLS + LayerNorm	$(B, 128)$
Linear(128→200)	$(B, 200)$

- CLS attends to **all 256 TPSAPU-state positions**
- Each position = one CNN spatial patch processed through the reservoir
- The transformer never sees pixels — it sees **spike/membrane histories**
- 4096→128 for ResCNN: 32× compression before any attention

What does CLS pay attention to?

CNN3-702k: L1 entropy ≈ 5.25 (max 5.55 for 256 tokens) — **near-uniform** over all 256 positions. The decoder reads the whole image rather than fixating on specific patches.

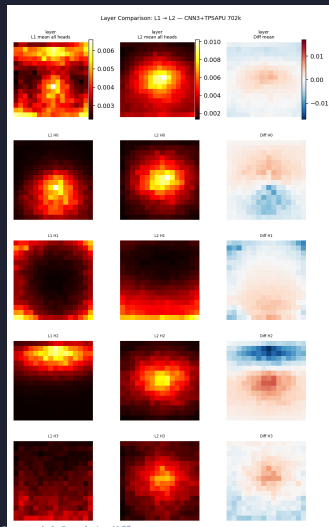
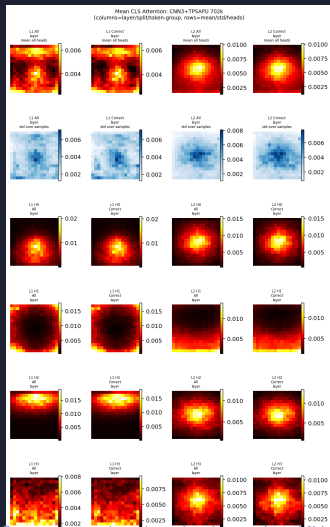
Head specialization: 4 heads focus on different image quadrants (bottom-right, top-right, left-center, right-center). Cosine similarity ≈ 0.3 – 0.6 between heads.

Layer refinement: L2 is slightly *more focused* on correct predictions (entropy -0.04 lower) — the second layer sharpens what L1 spread broadly.

Comparison to ViT

ViT processes *raw image patches* in its transformer. Here the transformer processes **spiking neural activity patterns** — one step removed from pixels.

CNN3-702k: Where the Transformer Looks (Aggregate, 500 Samples)



Key finding

CLS attention is broad and close to uniform.

No single spatial patch dominates the decoder decision.

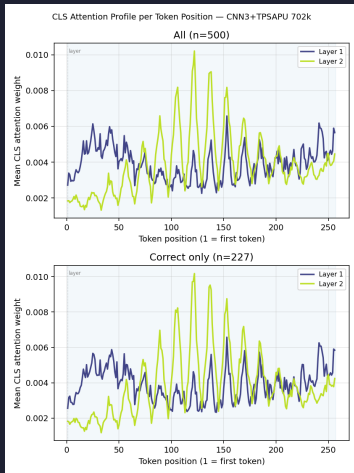
Some heads show weak preference for **corners and borders**.

Layer 2 mildly re-weights attention compared with Layer 1.

So the transformer mostly aggregates the full TPSAPU reservoir context, rather than selecting one patch.

Left: mean CLS attention, std, and per-head attention. Middle: Layer 1, Layer 2, and L2-L1 difference.

CNN3-702k: Attention Profile Over Token Order



CNN3 produces a 16×16 grid, so token positions 1, . . . , 256 correspond to spatial CNN patches in fixed scan order.

Continuation of the aggregate view

The previous slide showed *where* attention lands on the 2D grid. This plot shows the same attention as a 1D token-position profile.

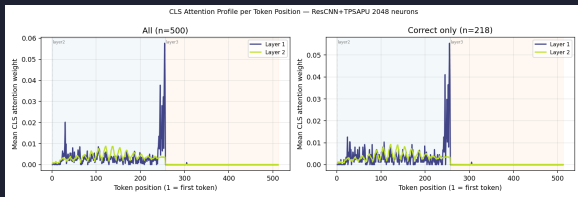
Layer 1: flatter curve, meaning attention is spread broadly across many tokens.

Layer 2: sharper peaks, meaning the second transformer layer becomes more selective.

The correct-only profile is very similar to the all-sample profile, so this is a stable behavior rather than an outlier effect.

Takeaway: CNN3-702k uses broad context, but Layer 2 slightly amplifies selected token regions instead of focusing on one dominant patch.

The Fun Fail: ResCNN Layer3 Was Never Normalized



CLS attention profile across all 512 token positions. Tokens 0–255 = res_cnn layer2. Tokens 256–511 = **layer3** (un-normalized).

What happened

res_cnn layer3 is a residual refinement: $\text{layer3} = \text{layer2} + \text{conv3}(\text{layer2})$

No LayerNorm added. Layer3 magnitudes are **much larger**. When fed into TPSAPU, many neurons immediately exceed threshold and **spike like crazy** — high-frequency noise rather than signal.

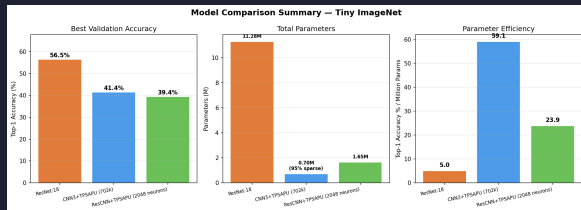
But the model still works!

The transformer **learned to ignore** tokens 256–511: L1 attention is **essentially zero** for all layer3 positions. L2 corrects by spreading attention more uniformly.

Top-1: **39.2%** despite half the tokens being useless. *Imagine the potential with normalized layer3 + more training!*

Scaling Up: Three Models on Tiny ImageNet (200 classes, 64×64)

	MNIST	Tiny ImageNet
Classes	10	200
Resolution	28 ²	64 ² , 3ch
CNN tokens	49	256 or 512
Taus	3	8
d_{res}	64	64 or 256
Neurons	192	512 or 2048
Training	20 ep	40–104 ep



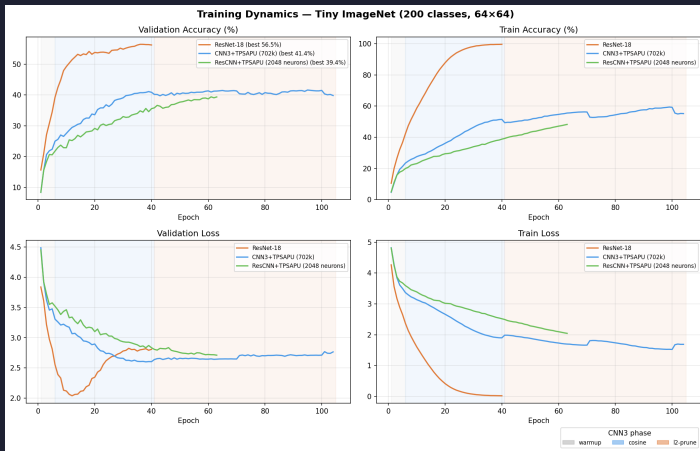
ResNet-18 baseline

11.28M params, tiny stem (3×3 , no maxpool), AdamW, 40 epochs. Strong spatial inductive bias.

CNN3+TPSAPU 702k

CNN3 encoder, TPSAPU (512 neurons, 8τ), Membrane-Transformer. 95% pruning target, 104 epochs.

Training Dynamics: All Three Models

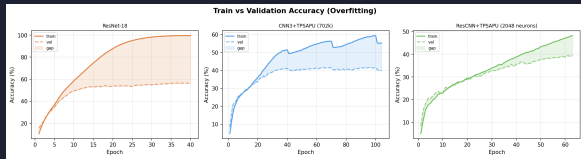


ResNet-18: fast convergence, strong overfit train 99% vs val 57%

CNN3-702k: l2-prune dip then recovery; less overfit

ResCNN-2048: 63/85 epochs; small batch slows convergence

Efficiency: 12× More Accuracy per Parameter



Train (solid) vs val (dashed). TPSAPU models overfit far less.

Model	Params	Top-1	Acc/M
ResNet-18	11.28M	56.7%	5.0
CNN3-702k	0.70M	41.6%	59.3
RCN-2048	1.65M	39.2%	23.8

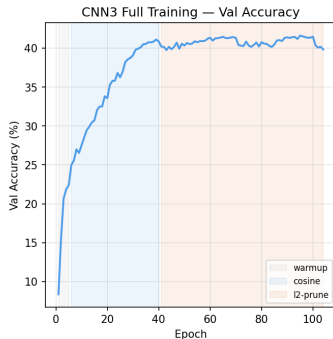
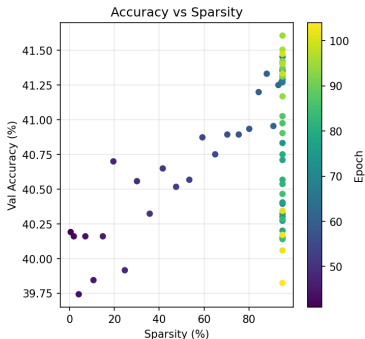
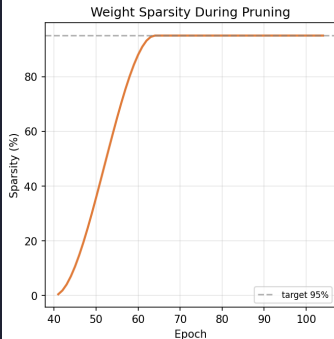
Parameter efficiency

CNN3-702k achieves **59.3 acc%/M params** vs 5.0 for ResNet-18 — **12× more efficient**

TPSAPU's implicit regularization (dropout, sparse recurrent, binary spikes) prevents the extreme overfit seen in ResNet-18 (train 99% → val 57%). The TPSAPU gap (train–val) stays <15pp.

95% Sparsity with Only Minor Accuracy Loss

CNN3+TPSAPU: Pruning Analysis



L2 pressure ramps over 24 epochs, then 6 epochs stabilize at the 95% mask.

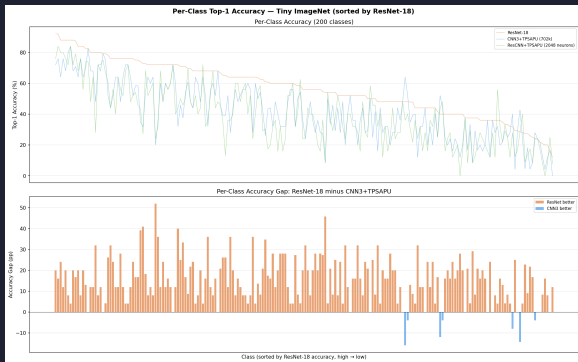
Accuracy drop

Dips during aggressive pruning then partially recovers in stabilization.

Key result

41.6% top-1 with only 5% of recurrent weights non-zero. Ideal for hardware.

Per-Class Analysis: 200 Classes, 4909 Validation Samples

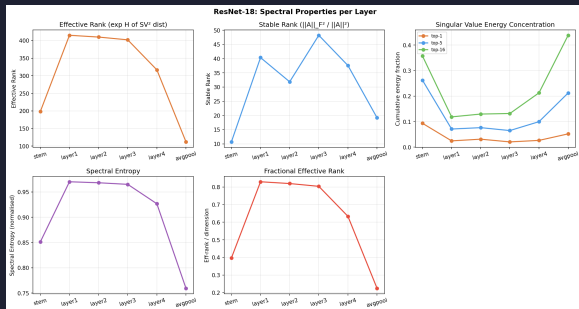


Classes sorted by ResNet-18 accuracy. Lower panel:
accuracy gap ResNet minus CNN3.

- All models fail on the *same* hard classes (fine-grained species, similar textures)
- Easy classes (goldfish, pizza, alp): all models $>85\%$
- CNN3-702k **beats ResNet-18 on specific classes** despite $16\times$ fewer params
- Top-5: ResNet 79.2% — CNN3 67.6% — ResCNN 65.4%
- TPSAPU top-5/top-1 ratio: $1.63\times$ vs $1.40\times$ for ResNet (more uncertain but top-5 competitive)

The per-class confusion structure is **similar across models** — difficulty is mostly in the data (64px thumbnails of 200 classes), not the architecture.

ResNet-18 Internal Analysis: Effective Rank per Layer



$$\text{Eff. rank} = e^H, H = -\sum_i p_i \ln p_i,$$

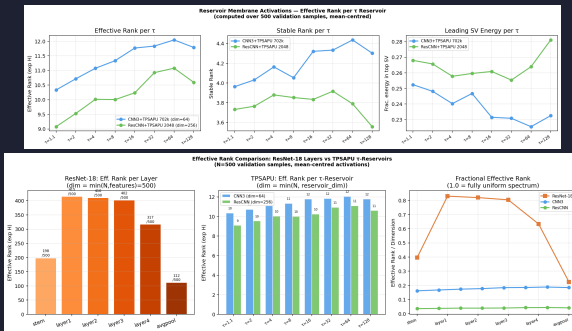
$$p_i = \sigma_i^2 / \|\sigma\|^2$$

Layer	Eff. Rank	Top-1 E
stem	198	9.4%
layer1	415	2.5%
layer2	410	3.1%
layer3	402	2.1%
layer4	317	2.7%
avgpool	112	5.2%

Middle layers span *almost the full sample space* (eff. rank 400/500). avgpool **bottlenecks** to 112 — strong overfit: the huge intermediate capacity maps to a narrow final representation.

TPSAPU is structurally different: the reservoir is a **fixed-dim bottleneck** that never expands — no intermediate overfit pathway.

TPSAPU Effective Rank: Redundant, Low-Dimensional, Still Working



Why it still matters

Redundant reservoir. The TPSAPU activations have much lower fractional effective rank than ResNet-18 layers. Even with a larger reservoir, the effective rank stays around 9–12.

Bloated matrix. This suggests that many neurons encode correlated directions. The recurrent matrix may be oversized: wider reservoirs do not automatically create independent features.

Still functional. This is not a complete failure mode. The decoder can still extract class signal from the compact subspace, so the reservoir acts like a strong regularizing bottleneck.

Future: FPGA & Neuromorphic Hardware

Why TP-SAPU maps naturally to hardware

- **Binary spikes** — multiply becomes wire-and-gate
- **95% sparse W_{rec}** — CSR format, 95% fewer ops
- **Streaming tokens** — one step at a time, unroll pipeline
- Shared W_{rec} across all τ s — one BRAM block
- Low-rank cross-reservoir ($r=16$) — tiny extra cost
- Fixed-point LIF membrane: 16-bit sufficient

Estimated hardware gains

- Dense CNN: all layers active every cycle
- TP-SAPU at 95% sparse + 10% spike rate: $\approx 0.5\%$ of multiply-adds remain
- Token-streaming: latency $\propto$ sequence length, not image width — natural for edge sensors
- FPGA prototype target: $<1\text{W}$ for 512-neuron model

Neuromorphic targets

- Intel Loihi 2 — LIF neurons, on-chip learning
- BrainScaleS-2 — analog membrane dynamics
- SpiNNaker 2 — massively parallel spike routing

Future: Event-Based Processing & Dynamic Vision Sensors

Why TP-SAPU is a natural fit for event cameras

- **DVS cameras** emit events $(x, y, t, \text{polarity})$ asynchronously
- Dense CNNs process fixed-size frames — **mismatched representation**
- TP-SAPU already processes a **token sequence** of variable length
- Events can be tokenized spatially ($\rightarrow$ CNN patches) or temporally ($\rightarrow$ time bins)
- **Sparse events** $\leftrightarrow$ **sparse spikes**: both architecturally aligned

Other temporal data streams

- **Audio**: mel-spectrogram patches $\rightarrow$ TP-SAPU $\rightarrow$ classify
- **Radar**: range-Doppler patches, temporal returns
- **Accelerometer/IMU**: activity recognition, window $\rightarrow$ tokens
- **EEG/LFP**: brain signal with biological neurons

Why multi-timescale matters especially here

Event streams have structure at many scales simultaneously: $\tau=1.1$ captures instantaneous edges, $\tau=128$ accumulates motion context over many frames. A single-state RNN must trade off one for the other. TP-SAPU holds **both at once**.

Future Work: Short, Medium, Long Term

Short term

- Finish ResCNN-2048 training (22 epochs remaining)
- **Normalize res_cnn layer3** — obvious fix, expect >43%
- Apply neuron pruning (already implemented, never used)
- Transformer decoder sweep on MNIST
- Tau ablation: how much accuracy do you lose with $N_\tau=1, 2, 4$?
- Spike rate / accuracy Pareto curve

Medium term

- Full ImageNet 1k with deeper encoder (cnn5)
- Learned τ values (differentiable time constants)
- DVS event-camera tokenizer
- Adaptive token ordering: attend to salient regions first

Architecture improvements

- **BatchNorm/LayerNorm after each encoder layer** — the res_cnn failure shows this matters
- Pre-LN transformer (more stable training)
- Cross-attention decoder instead of CLS: let decoder query which reservoir state it needs
- Hierarchical reservoir: coarse-then-fine token processing
- Online/continual learning via STDP on top of the spiking backbone

The most impactful next step: fix layer3 normalization, finish training, and run the full-split evaluation. Based on the attention analysis, this single fix could close ~ 5 pp of the accuracy gap.

Pros & Cons of the TP-SAPU Architecture

Strengths

- **12× parameter efficiency** vs ResNet-18 on ImageNet
- Multi-timescale memory with *zero* extra recurrent cost
- 95% recurrent sparsity with minimal accuracy loss
- **Near-zero overfit** — sparsity + spikes regularize implicitly
- Modular: encoder, backbone, decoder all swappable
- Hardware-ready: binary, sparse, streaming, shared weights
- Scales from 99.4% MNIST to competitive ImageNet

Weaknesses

- **15pp accuracy gap to ResNet-18** on ImageNet
- Sequential token loop — hard to fully parallelize
- Surrogate gradient is imperfect
- Small batch needed for large models (slow training)
- No BatchNorm in backbone — sensitive to encoder normalization
- Training time: 63 epochs = 6 days (ResNet: 40ep = 1.3h)
- **2048-neuron model not yet converged**
- Feature interpretation harder than standard CNNs

Conclusion

What we showed

- TP-SAPU is a **validated architecture**: 99.41% on MNIST, 41.6% on Tiny ImageNet
- **Binary spikes + shared topology + 8 timescales** outperform single-state RNNs in parameter efficiency
- 95% recurrent sparsity is achievable without catastrophic accuracy loss
- Transformer decoder learns to read the reservoir **broadly** — no single patch dominates
- Even with half the tokens unusable (un-normalized layer3), the transformer adapts and **ignores them**

Metric	Value
MNIST top-1	99.41%
ImageNet top-1 (702k)	41.6%
ImageNet top-5 (702k)	67.6%
Params vs ResNet	16× fewer
Acc/M vs ResNet	12× better
Recurrent sparsity	95%
Spike rate	~10%

THANK YOU!